REFERENCE GUIDE

dockwatch FAQ

Answers to the questions that come up most — architecture, updates, permissions, and what's not supported yet.

VERSION	CATEGORIES	GENERATED
0.7.0	**6**	**2026-08-12**

Architecture & Deployment

Q **Does it work across multiple Docker hosts?**

Yes, via Portainer environments — one dockwatch instance can monitor containers across many hosts through Portainer's API, plus the local Docker socket.

Q **Is this agent-based?**

No. Local mode uses the Docker socket directly. Remote hosts go through Portainer's existing API. Nothing to install on target hosts.

Q **What registries are supported?**

Docker Hub, GitHub Container Registry (ghcr.io), LinuxServer.io (lscr.io), and Codeberg. Floating tags (`latest`, `edge`, `dev`, `nightly`) are handled via digest comparison, not just tag string matching.

Filtering & Dashboard

Q **When I switch between Local / Portainer / All, does it re-pull from Docker Hub?**

No. Filter switching is purely client-side — it re-filters already-loaded results in memory. No API calls, no registry checks. A check only fires on initial page load, manual Refresh button click, or the background schedule.

Q **How does dockwatch know if a container is Portainer-managed or local?**

Containers deployed via Portainer stacks have compose labels pointing to `/data/compose/{id}/`. Dockwatch detects these automatically and tags them as `source=portainer`, even when discovered via the local Docker socket.

Q **Why do I see the same containers under both Local and Portainer?**

If your Portainer instance connects to the same Docker daemon as your local socket, all containers are visible to both. Dockwatch uses label-based detection to assign each container to the correct source — the dashboard filter groups them accordingly.

Updates

Q **Can it update anything, or just compose stacks?**

- **Local Docker**: compose-managed and plain `docker run` containers are both supported.
- **Portainer**: compose-managed (stack-deployed) containers only. Non-compose Portainer containers can be restarted or deleted but not pull-updated.

What happens when I click Update on a Portainer stack container?

- Builds update plan — verifies the container is eligible
- Finds the stack via Portainer API (`GET /api/stacks`)
- Reads the current stack compose file (`GET /api/stacks/{id}/file`)
- Rewrites the target service's `image:` line (e.g. `nginx:1.25` → `nginx:1.27`)
- Redeploys via Portainer (`PUT /api/stacks/{id}`) with `pullImage: true`

The redeploy step blocks until Portainer finishes pulling the new image and recreating the container, which can take a while for large images. This uses a longer, separate timeout (`portainer.deploy_timeout` , default 120s) than other Portainer calls — raise it in Settings if large-image redeploys still time out. If a redeploy does time out, Portainer usually completes it server-side anyway; check the stack's status in Portainer before retrying to avoid a duplicate/conflicting deploy.

If I update one service in a multi-service stack, does it restart everything?

No. Only the target service is affected:

- **Local compose**: runs `docker compose pull <service>` and `docker compose up -d <service>` — scoped to that one service.
- **Portainer stack**: only the target service's image line is rewritten. Portainer diffs the compose content and recreates only the changed service. Sibling services stay running untouched.

Can I delete containers and images through dockwatch?

Yes. Delete requires the `delete_containers` permission (separate from `update_containers`). Works for both local Docker and Portainer-managed containers with a confirmation prompt. Image deletion is supported for local containers only (Portainer-sourced image deletion requires direct Docker socket access). All deletions are logged to the audit trail.

What if an update breaks something?

Rollback button reverts to the last known-good tag for compose-managed containers (one click from the History panel). Plain-mode local updates auto-rollback on failure during the update itself — if the replacement container fails to start, the original is restored.

Scheduling & Caching

Does dockwatch check for updates automatically, or only when I click Refresh?

Both. The web server runs a background scheduled check on the configured interval (default 300 seconds + jitter). It keeps the results cache warm and broadcasts fresh data to connected dashboards via WebSocket. You can also click Refresh anytime for an immediate check.

How does dockwatch avoid Docker Hub rate limits (429 errors)?

Three in-memory caches reduce API calls:

Cache	TTL	What
Tag list	5 min	`hub.docker.com/v2/repositories/ ... /tags`
Manifest digest	60 sec	`registry-1.docker.io/v2/ ... /manifests/{tag}`
Auth token	60 sec	`auth.docker.io/token`

Rapid successive checks (page refresh, filter switching, auto-refresh) reuse cached data within the TTL window instead of hitting Docker Hub.

The dashboard shows stale results — how do I refresh?

Click the **Refresh** button in the toolbar, or wait for the next scheduled background check. Filter switching (Local/Portainer/All) does not trigger a refresh — it re-filters the current results client-side.

Permissions & Security

What permissions are available?

Six fixed permissions: `view_containers`, `update_containers`, `delete_containers`, `scan_containers`, `manage_settings`, `manage_users`. Combinable into custom roles. Built-in roles: `admin` (all six) and `viewer` (view only).

What's the trust boundary?

`manage_settings`, `update_containers`, and `delete_containers` are effectively admin-equivalent — all three can reach the host's Docker daemon indirectly. Only grant these to people you'd trust with direct `docker.sock` access.

Limitations

What's NOT supported yet?

- Full stack recreate with network/volume changes via Portainer (only image tag updates are supported)
- Kubernetes environments
- Rollback for Portainer-managed containers (local compose only)
- Image deletion for Portainer-sourced containers

Does it work with Docker Swarm?

Not currently. Portainer integration targets standalone Docker environments.